

Tổng Hợp Kiến Thức

1. Cấu hình môi trường với `.env`

- **Mục đích:** Tách biệt cấu hình (API URL, keys) khỏi code, dễ chuyển đổi môi trường.
- **Cách làm:**
 - Tạo file `.env` ở thư mục gốc.
 - Thêm biến môi trường (dùng tiền tố `VITE_` cho Vite):

```
VITE_API_BASE_URL=http://localhost:5000/api
```
 - Truy cập trong code: `process.env.VITE_API_BASE_URL`.
- **Lưu ý:** Không commit `.env` lên Git nếu chứa thông tin nhạy cảm.

2. Tối ưu gọi API với Axios Instance

- **So sánh Fetch API vs Axios:**
 - **Fetch API:** Built-in, không cần cài đặt, nhưng cú pháp dài, không tự parse JSON, xử lý lỗi thủ công.
 - **Axios:** Cần cài (`npm install axios`), cú pháp ngắn gọn, tự parse JSON, xử lý lỗi tốt hơn.
- **Tạo Axios Instance:**

```
// src/api/axiosClient.js
import axios from 'axios';

const axiosClient = axios.create({
  baseURL: process.env.VITE_API_BASE_URL, // Từ .env
  headers: { 'Content-Type': 'application/json' },
});

export default axiosClient;
```

- **Sử dụng:**

```
const response = await axiosClient.get('/todos');
console.log(response.data);
```

3. Redux Thunk - Xử lý bất đồng bộ

- **Mục đích:** Tách logic gọi API khỏi component, tái sử dụng, dễ test.
- **Tạo Async Thunk:**

```
import { createAsyncThunk } from '@reduxjs/toolkit';
import axiosClient from '../api/axiosClient';

export const fetchTodos = createAsyncThunk(
  'todos/fetchTodos', // Action type
  async () => {
    const response = await axiosClient.get('/todos');
    return response.data; // Payload cho fulfilled
  }
);
```

- **Xử lý trong Slice:**

```
const todoSlice = createSlice({
  name: 'todos',
  initialState: { todos: [], loading: false, error: null },
  extraReducers: (builder) => {
    builder
      .addCase(fetchTodos.pending, (state) => {
        state.loading = true;
        state.error = null;
      })
      .addCase(fetchTodos.fulfilled, (state, action) => {
        state.loading = false;
        state.todos = action.payload;
      })
      .addCase(fetchTodos.rejected, (state, action) => {
        state.loading = false;
        state.error = action.error.message;
      });
  },
});
```

4. Kết nối Component với Redux Thunk

- **Dispatch Thunk trong Component:**

```
// src/components/ToDoList.jsx
import { useEffect } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { fetchTodos } from '../redux/slices/todoSlice';

function ToDoList() {
  const dispatch = useDispatch();
  const { todos, loading, error } = useSelector((state) => state.todos);

  useEffect(() => {
    dispatch(fetchTodos());
  }, [dispatch]);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <ul>
      {todos.map((todo) => (
        <li key={todo.id}>{todo.text}</li>
      ))}
    </ul>
  );
}
```

- **Các Hook Redux:**

- `useSelector` : Lấy state từ store (`state.todos.todos`).
- `useDispatch` : Dispatch actions/thunks.